



Conversor de Notación

Infija a Postfija y Prefija en Rust

Juan Morales C.I.: 30916541

Omaira Rodriguez C.I.: 15351394

Keiber Mendina C.I.: 31401788

Análisis de Sistemas

Lenguajes de Programación(C7)

El Problema y Restricciones

🔍 Enunciado del Proyecto

Dada una expresión aritmética en notación infija y una tabla de símbolos con jerarquía de operadores, se debe tokenizar la expresión y transformarla de manera óptima hacia representaciones postfijas y prefijas.

- Tokenización precisa en un único paso.
- Evaluación de operadores según prioridad configurada.
- Exportación en representaciones normalizadas.

🚫 Restricciones Estrictas de Memoria

- Uso obligatorio de **arreglos de tamaño fijo**.
- Prohibición estricta de colecciones dinámicas (Vec, HashMap).
- Expresiones limpias sin espacios ni paréntesis redundantes.
- Operandos acotados a caracteres de la A a la Z.
- Operadores básicos soportados: +, -, *, /.

Tipos de Notaciones

Notación	Descripción Conceptual	Ejemplo: A+B*C
Infija	El operador se ubica entre sus operandos correspondientes. Comprensión natural humana.	A + B * C
Postfija (RPN)	El operador sigue a sus operandos correspondientes. Usado en pilas y máquinas virtuales.	A B C * +
Prefija (Polaca)	El operador precede a sus operandos correspondientes. Evaluación directa recursiva.	+ A * B C

Ventaja de RPN

Facilidad de computación a bajo nivel. Permite omitir totalmente los paréntesis y procesar las variables directamente con pilas LIFO.

Ventaja de Prefija

Permite la construcción de árboles de sintaxis abstracta (AST) de forma recursiva y es el estándar sintáctico de Lisp.

Algoritmo

☰ Conversión a Postfija

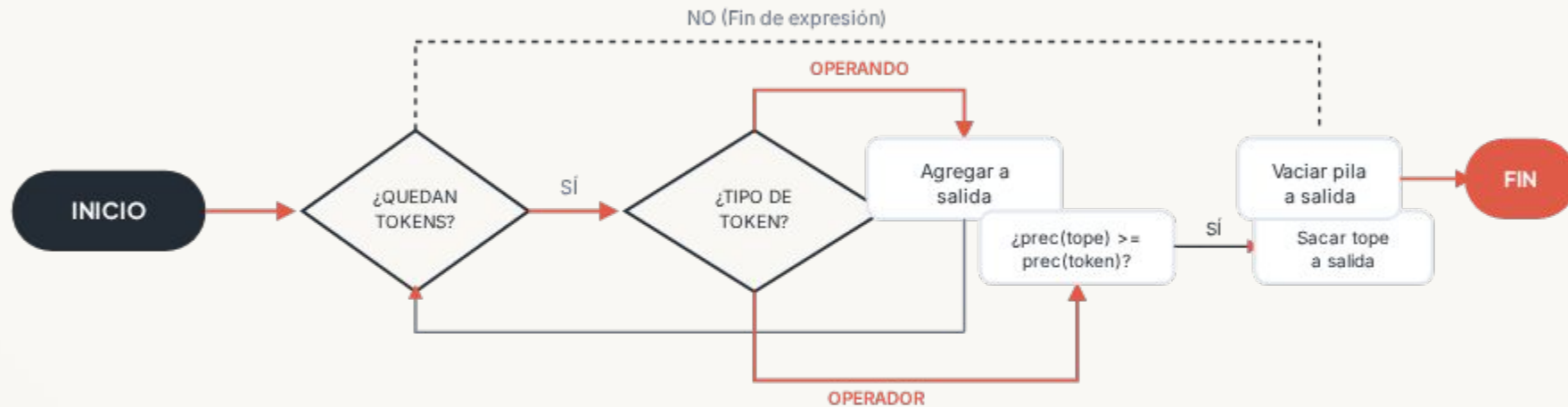
```
para cada token de izquierda a derecha:  
  si es OPERANDO → agregar a salida  
  si es OPERADOR:  
    mientras pila no vacía  
      y  $\text{prec}(\text{tope}) \geq \text{prec}(\text{token})$ :  
        sacar tope → salida  
    meter token en pila  
  
al final: vaciar pila → salida
```

🔄 Adaptación para Prefija

Para obtener la expresión prefija utilizando la lógica del Shunting Yard de Dijkstra, aplicamos una refinada técnica de transformaciones reversas:

- **Inversión inicial:** Invertir el orden lógico de los tokens de entrada.
- **Shunting Yard Estricto:** Ejecutar la ordenación aplicando comparación de precedencia estricta ($>$ en lugar de $\geq$).
- **Inversión final:** Invertir el flujo resultante para obtener la secuencia prefija correcta.

Diagrama de Flujo: **Algoritmo**



Estructuras de Datos Estáticas



Tabla de Símbolos

Coordinación en base a arreglos paralelos con capacidad fija (máx. 50 símbolos).

- `simbolos: [char; 50]`
- `tipos: [Tipo; 50]`
- `precedencias: [i32; 50]`
- `cantidad: usize`



Tokens de Expresión

Estructura temporal que contiene la expresión parseada lista para procesar (máx. 100).

- `tokens_simb: [char; 100]`
- `tokens_tipo: [Tipo; 100]`
- `tokens_prec: [i32; 100]`
- `cantidad_tok: usize`



Pila de Operadores

Pila local LIFO optimizada para ejecución del algoritmo Shunting Yard.

- `pila_simb: [char; 100]`
- `pila_prec: [i32; 100]`
- `tope: usize`

Enumeración Tipo en Rust

</>Modelado de Tipos

En Rust, aprovechamos las propiedades del enum nativo para restringir los tipos permitidos dentro de la gramática del analizador sintáctico.

"Derive `Clone`, `Copy` y `PartialEq` garantiza la manipulación por valor eficiente de las variantes dentro de los arreglos estáticos sin requerir punteros ni gestores dinámicos."

```
#[derive(Clone, Copy, PartialEq)]
enum Tipo {
    Operando,
    Operador,
}
```

Funciones del Programa

Función	Propósito del Módulo
leer_archivo	Abre y parsea la expresión y la tabla de símbolos del archivo plano de entrada.
buscar_simbolo	Consulta lineal sobre la estructura estática para retornar el tipo y jerarquía del carácter.
tokenizar	Genera los arreglos paralelos estructurados a partir de la cadena original leída.
mostrar_tabla_tokens	Herramienta visual para imprimir la tabla de análisis en la consola estándar.
infija_a_postfija	Algoritmo clásico Shunting Yard para conversión a notación Polaca Reversa.
infija_a_prefija	Invierte el flujo y procesa el algoritmo con ordenamiento estricto para generar notación polaca.

Formato de Entrada

Archivo de Texto (entrada.txt)

```
EXPRESION=A+B*C  
A, OPERANDO, 0  
B, OPERANDO, 0  
C, OPERANDO, 0  
+, OPERADOR, 1  
-, OPERADOR, 1  
*, OPERADOR, 2  
/, OPERADOR, 2
```

Reglas de Estructura

- **Fila 1:** Debe iniciar exactamente con EXPRESION= seguido de la cadena a procesar.
- **Filas subsiguientes:** Representación del símbolo mediante el formato delimitado por comas simbolo,tipo,precedencia.
- **Robustez:** El parser descarta líneas vacías o en blanco automáticamente.

Ejecución del Programa

```
bash - rust_converter_cli
```

```
x
```

```
# Compilación y ejecución usando el gestor de paquetes Cargo
```

```
$ cargo run --release
```

```
# Ejecución especificando un archivo personalizado de entrada
```

```
$ cargo run --release -- /ruta/hacia/expresion_personalizada.txt
```

```
# Ejecución directa del binario autocontenido optimizado
```

```
$ ./target/release/conversor_notacion entrada.txt
```

Consola de Salida

stdout - Terminal de Resultados

≡ Tabla de tokens ≡

Token	Tipo	Precedencia
-------	------	-------------

A	OPERANDO	0
---	----------	---

+	OPERADOR	1
---	----------	---

B	OPERANDO	0
---	----------	---

*	OPERADOR	2
---	----------	---

C	OPERANDO	0
---	----------	---

Expresion infija: A+B*C

Expresion postfija: ABC*+

Expresion prefija: +A*BC

Gestión de Excepciones

Situación de Entrada Anómala	Mensaje de Error en Terminal
Archivo de entrada no disponible	No se pudo abrir 'archivo': [Detalles del OS]
Archivo de texto vacío	Archivo vacío
Estructura de primera línea inválida	La primera línea debe comenzar con EXPRESION=
Línea de símbolo con formato roto	Formato inválido en línea: '...'
Clase de símbolo inválida	Tipo desconocido '...' en línea '...'
Expresión aritmética vacía	la expresión está vacía
Expresión excede límites (>100 caracteres)	expresion demasiado larga (max 100)

Traza del Algoritmo: A+B*C

→Postfija

[+, *]

Token	Acción Realizada	Pila	Salida
A	OPERANDO → Salida	[]	A
+	OPERADOR → Pila	[+]	A
B	OPERANDO → Salida	[+]	AB
*	prec(*) > prec(+) → Pila	AB	
C	OPERANDO → Salida	[+, *]	ABC
—	Vaciar pila a salida	[]	ABC*+

←Prefija

Token Inv.	Acción Realizada	Pila	Salida
C	OPERANDO → Salida	[]	C
*	OPERADOR → Pila	[*]	C
B	OPERANDO → Salida	[*]	CB
+	prec(+) no > prec(*) → Pila	[+, *]	CB
A	OPERANDO → Salida	[+, *]	CBA
—	Vaciar e Invertir	[]	+A*BC

Verificación de Casos

Pruebas exhaustivas realizadas en el motor de ejecución para validar precedencias y comportamientos de asimilación sintáctica de izquierda a derecha y viceversa:

Expresión Infija de Entrada	Resultado Postfijo (RPN)	Resultado Prefijo (Polaca)
A + B * C	A B C * +	+ A * B C
A * B + C	A B * C +	+ * A B C
A + B - C	A B + C -	- + A B C
A * B / C	A B * C /	/ * A B C
A + B + C	A B + C +	+ + A B C

Conclusiones del Proyecto

- **Seguridad sin Sobrecarga:** Rust demuestra que es viable estructurar analizadores léxicos y sintácticos de alta velocidad y nulo overhead de recolección de basura con seguridad de memoria por defecto.
 - **Determinismo Estricto:** Las limitaciones autoimpuestas por el uso de arreglos de tamaño fijo imponen una disciplina que optimiza drásticamente el espacio de memoria, eliminando fugas potenciales y fragmentaciones.
 - **Eficiencia Algorítmica:** Shunting Yard se consolida como un algoritmo elegante con una complejidad temporal lineal de $O(n)$ y una complejidad espacial acotada de $O(n)$.
 - **Simetría Lógica:** La solución implementada aprovecha la elegante simetría matemática entre las expresiones polaca y polaca inversa para reutilizar casi en su totalidad el algoritmo principal mediante inversiones simples de flujo.
-